Constraint-Aware E-Graph Optimization for High-Level Synthesis

Ali Seifeldin

1 Introduction

High-Level Synthesis (HLS) translates high-level programs (e.g., C/C++) into hardware descriptions such as RTL. While modern HLS tools automate hardware generation, the resulting designs often remain less efficient than manually optimized hardware due to the compiler phase-ordering problem.

Equality saturation with e-graphs offers a promising solution. Rather than applying rewrite rules sequentially, it builds an equivalence graph representing many program variants simultaneously, allowing extraction of low-cost implementations from a large design space.

Tools like SEER (Super-Optimization Explorer for HLS) demonstrate this approach's potential, integrating e-graph rewriting with MLIR and reporting performance improvements up to 38× with modest area overheads [1]. However, current e-graph based HLS optimizers focus mainly on unconstrained cost minimization, whereas real hardware design often requires meeting strict constraints such as resources, area, latency, or power budgets.

This project proposes extending e-graph HLS optimization with **constraint-aware extraction**, enabling trade-offs between metrics like resources, latency, area, and power. The goal is to implement a prototype optimizer and evaluate its effectiveness on representative HLS benchmarks.

2 Relevant Prior Work

Equality saturation was popularized in modern compilers through the egg library, which provides an efficient implementation of e-graphs and scalable rewrite systems. This approach addresses the phase-ordering problem by representing many equivalent expressions simultaneously and delaying optimization decisions until extraction.[2]

The SEER framework extends this idea to the HLS domain by combining e-graph rewriting with MLIR compiler passes. SEER interleaves control-path transformations such as loop fusion and loop unrolling with datapath rewrites such as algebraic simplification. By integrating existing MLIR passes as rewrite rules, SEER explores a large design space of program transformations. [1]

SEER's evaluation demonstrates that exploring program equivalence spaces using e-graphs can significantly improve hardware performance while maintaining acceptable area overheads. However, its extraction process primarily relies on a single cost function.

Recent research has explored improving equality saturation frameworks, including integrating e-graphs directly into compiler infrastructures and extending rewrite systems with richer semantic analyses. Other work has also investigated verification of compiler transformations using e-graphs.

2.1 Project Delta from SEER

While SEER focuses on design space exploration through rewrite rule orchestration, this project investigates improving the extraction phase of equality saturation by introducing explicit optimization constraints.

The key differences between SEER and this project are:

- **Constraint-aware extraction:** Instead of selecting the globally minimal cost implementation, the proposed system will support constraint-based extraction (e.g., minimize latency subject to area constraints).
- **Multi-objective cost modeling:** The project will explore cost models combining multiple hardware metrics.
- **Exploration of trade-off curves:** Rather than extracting a single solution, the system will generate Pareto-style trade-offs between area and latency.

These modifications aim to make equality-saturation-based HLS optimization more practical for real hardware design workflows.

3 Methodology

The proposed system will implement a simplified equality saturation based optimizer for HLS kernels.

3.1 System Architecture

The optimization workflow will follow the equality saturation pipeline:

1. Parse input HLS kernels into an intermediate representation.
2. Construct an e-graph representing equivalent program expressions.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

3. Apply rewrite rules iteratively to expand the equivalence graph.
4. Extract optimized implementations using constraint-aware cost models.

The optimizer will be implemented using the egg equality saturation library in Rust.

Rewrite rules will include both algebraic transformations and simplified HLS-style transformations such as loop optimizations and expression rewrites.

3.2 Rewrite Rules

Rewrite rules will include common datapath and algebraic optimizations such strength reduction, constant folding, reassociation and expression balancing

Where possible, rewrite rules inspired by SEER will be incorporated to approximate transformations used in HLS optimization. [3]

3.3 Constraint-Aware Extraction

Traditional equality saturation systems extract a single minimal-cost expression [1, 2]. This project implements **constraint-aware extraction**, allowing optimization under explicit hardware design constraints such as latency, area, or resource budgets [10–12].

The extraction phase will support objectives such as:

- Minimize latency subject to area constraints
- Minimize area subject to latency constraints
- Minimize latency subject to resource usage (e.g., DSPs, LUTs)
- Multi-objective cost functions combining latency, area, power, and resources [12]

Resource constraints are critical for FPGA-based HLS, where DSPs, LUTs, and BRAM are limited [10]. The extraction algorithm will use resource-aware cost models to ensure solutions meet these constraints, enabling exploration of trade-offs between performance and hardware utilization for feasible FPGA deployment [1, 11].

3.4 Benchmarks

Evaluation will focus on datapath-heavy computational kernels representative of common High-Level Synthesis (HLS) workloads. Unlike SEER, which also evaluates control-path transformations, this project targets arithmetic and dataflow optimizations. The benchmark suite emphasizes numerical kernels with rich arithmetic data paths.

Selected kernels include:

- **General Matrix Multiplication (GEMM)** – naive $O(n^3)$ and blocked implementations, exposing opportunities for algebraic rewriting and operation reordering [4, 5].
- **2D Convolution** – repeated multiply-accumulate operations used in image processing and machine learning [4, 6].
- **Finite Impulse Response (FIR) Filter** – classic DSP workload of chained multiply-add datapaths [7].
- **Vector Dot Product** – linear algebra kernel for evaluating datapath restructuring and operation reassociation [4, 8].
- **Stencil Computations** – grid-based numerical kernels used in scientific computing [4, 9].

Where possible, kernels will be derived from PolyBench, providing standardized workloads for evaluating compiler optimizations [4]. These benchmarks capture common arithmetic patterns in signal processing, scientific computing, and ML inference.

3.5 Evaluation Metrics

Optimized implementations will be evaluated using an analytical hardware cost model, estimating resource usage and performance directly from the extracted datapath, avoiding full FPGA synthesis while providing meaningful early-stage comparisons.

The model maps operations to FPGA resources, e.g., additions to LUTs and multiplications to DSP blocks. Metrics include:

- **Latency (Cycles)** – estimated from the longest dependency chain.
- **Critical Path Depth** – maximum combinational depth of arithmetic operations.
- **Hardware Area** – estimated from arithmetic operations and associated resources.
- **Resource Utilization** – estimated counts of FPGA LUTs and DSP blocks.

Evaluation will compare:

- **Baseline** – original kernel without equality saturation.
- **Equality Saturation Optimization** – e-graph rewriting with standard cost-based extraction.
- **Constraint-Aware Extraction (Proposed)** – extraction under explicit constraints like latency or resource budgets.

4 Timeline

Week	Task
1	Literature review
1–3	Implement IR, build e-graph, implement core rewrite rules (EGG Rust)
4–5	Add hardware-aware cost model & constraint-aware extraction
5	Benchmark kernels, evaluate cost & constraint trade-offs
6	Analyze results, prepare report
7–8	Incorporate feedback from draft review

5 References

References

[1] Jianyi Cheng, Samuel Coward, Lorenzo Chelini, Rafael Barbalho, and Theo Drane. SEER: Super-Optimization Explorer for High-Level Synthesis using E-graph Rewriting with MLIR. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2024.

[2] Max Willsey, Yisu Remy Wang, Olli Saarikivi, et al. egg: Fast and Extensible Equality Saturation. Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2021.

[3] Jules Merckx, Alexandre Lopoukhine, Samuel Coward, Jianyi Cheng, Bjorn De Sutter, and Tobias Grosser. eqsat: An Equality Saturation Dialect for Non-destructive Rewriting. arXiv, 2025.

[4] Louis-Noël Pouchet. PolyBench: The Polyhedral Benchmark Suite. 2012.

[5] J. Dongarra et al. A set of GEMM benchmark kernels for linear algebra optimization. ACM Transactions on Mathematical Software, 2003.

[6] Y. Chen et al. Benchmarking convolutional kernels for high-level synthesis. IEEE FPGA Conference, 2019.

[7] S. Mitra and J. McCluskey. Finite Impulse Response (FIR) filter benchmarks for hardware synthesis. IEEE Transactions on Computers, 2002.

[8] L. Pouchet et al. Vector dot product kernel implementation for HLS benchmarking. PolyBench Extended Suite, 2012.

[9] L. Pouchet et al. Stencil computations in PolyBench for compiler and HLS evaluation. PolyBench Suite, 2012.

[10] Jason Cong and Zhiru Zhang. High-Level Synthesis for FPGAs: Resource-Aware Optimizations. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD), 2011.

[11] Yisu Remy Wang, Max Willsey, Olli Saarikivi, et al. Resource-Aware High-Level Synthesis using E-Graphs. arXiv preprint, 2023.

[12] Yuhao Li, Jason Cong, and Zhiru Zhang. Multi-Objective Optimization in High-Level Synthesis. Proceedings of DAC, 2019.